

Lab 2: Batch Geoprocessing and Data Interrogation

Overview

Over the last few weeks, we have introduced:

1. **batch processing**, which allows us to write one tool call, then loop it over everything in a workspace instead of running it by hand
2. **interrogation**, where we never trust a file's shape, coordinate system, or fields until you've used `Describe` or `ListFields` directly.

This lab has three tasks. Task A builds an inventory of a workspace. Task B batch-buffers everything inside the workspace. In Task C, you will answer factual questions about the data *programmatically*.

Download the template notebook, `lab02_template.ipynb`, before you start. It has one markdown header and one seeded code cell per task below, in order — Task B assumes Task A's workspace setup, and Task C requires both, so do your work inside the template in order and submit the completed file.

Data: All are present in the course repostory. The data you'll need are:

- `ohio_counties.shp`
- `portage_munis.shp`
- `ohio_state_parks.shp`- `-streams_portage.geojson` file
- `ohio_precip_2025.csv`, all sitting directly in that folder

Warm-up: one row by hand

Before you write a loop that performs the following work for every feature class, investigate what the loop body needs to produce for a single one. Here's an example of it for `ohio_counties.shp`, worked by hand:

```

fc = "ohio_counties.shp"
desc = arcpy.da.Describe(fc)
shape_type = desc["shapeType"]
crs_name = desc["spatialReference"].name
count_result = arcpy.management.GetCount(fc)
feature_count = int(count_result[0])

print(f"{fc:<22} {shape_type:<10} {crs_name:<28} {feature_count} features")

```

The single `print` line is one row of Task A's table. For Task A, you'll need a loop that walks every feature class `arcpy.ListFeatureClasses()` finds. Therefore, the same four lines above (`desc`, `shape_type`, `crs_name`, `feature_count`) will become the body of that loop.

Task A: Inventory script (22 points)

Set `arcpy.env.workspace` to a sensible location and set `arcpy.env.overwriteOutput = True`.

Use `arcpy.ListFeatureClasses()` to return every feature class `arcpy` can see in that workspace. Loop over the result and, for each feature class, print one row of a table with these four columns:

- **name** — the feature class's filename,
- **shapeType** — from `Describe`,
- **CRS name** — the coordinate reference system's name, also from `Describe`,
- **feature count** — the number of features it contains.

You already know how to pull `shapeType` and a CRS name out of `arcpy.da.Describe()` from week 4 (refer to our in-class notebooks if you need help).

Feature count is new. `arcpy.management.GetCount()` takes a feature class and returns a `Result` object (not a plain number) so wrap it in `int(...)` to get something you can print, store, or compare. For example...

```

count_result = arcpy.management.GetCount("ohio_counties.shp")
feature_count = int(count_result[0])
print(feature_count)

```

Put those pieces together inside your loop over `ListFeatureClasses()`, and print one aligned row per feature class. In this case, use an f-string with `:<` width specifiers:

```
print(f"{fc:<22} {shape_type:<10} {crs_name:<28} {count} features")
```

Do not hardcode the feature class names, shape types, or counts. Your loop must discover everything from the workspace itself. If a file were added to or removed from the folder, your code should print a correspondingly different table without you touching it.

Expected output looks like: one aligned row per feature class arcpy finds in the workspace (three rows for this dataset) each row showing a filename, a shape type word, a CRS name, and a feature count, in that order.

Task B: Batch buffer (30 points)

Buffer **every polygon feature class** in the workspace at **two different distances of your choosing**. Pick two values that make sense for these data and say what you picked and why, in one sentence, in a markdown cell.

Reuse the derived-output-name pattern from our in-class work: strip the input's extension, and build the output name with an f-string that incorporates the distance, (e.g. something in the shape of `f"{base_name}_{distance_label}.shp"`). You'll need to use a nested loop — one loop over your two distances, one loop over the feature classes — so that every feature class gets buffered at both distances (six output files total, from three feature classes and two distances).

Hint: the derived-name pattern

“Derived” means the output name is *built from* the input's name and the current distance, not typed once per file. A possible shape to follow:

```
base_name = fc.replace(".shp", "")
out_name = f"{base_name}_{distance}mi.shp"
```

Use two checks to “guard” the execution in two ways:

- `arcpy.env.overwriteOutput = True`
- `arcpy.Exists()`, checked before each `Buffer` call, so a re-run *skips* work that's already done instead of silently redoing it. Print a `"skipping ... -- already exists"` line when this happens, same as the pattern in-class.

Hint: the Exists guard

Check *before* you call `Buffer`, not after. After all, the point is to skip the call entirely, not to run it and then notice:

```
if arcpy.Exists(out_name):
    print(f"skipping {out_name} -- already exists")
else:
    arcpy.analysis.Buffer(fc, out_name, f"{distance} Miles", dissolve_option="NONE")
```

This has to sit *inside* both loops, immediately before the `Buffer` call, so it's checked for every one of the six planned outputs.

Leave the dissolve option unset (or explicitly "NONE") — you want one buffer polygon per input feature, not one merged shape per feature class.

After each successful buffer, print a one-line confirmation using `GetCount` on both the input and the output. For a non-dissolved buffer, those two counts should always match. If they don't, something is wrong with your dissolve setting.

Expected output looks like: up to six confirmation lines (fewer on a re-run, replaced by “skipping...” lines), each naming an output file and showing two feature counts that match. Also, somewhere before or after the loop, one markdown sentence naming your two chosen distances and why.

Task C: Interrogation memo (19 points)

Answer each of the five questions below with a code cell (the code, not you, should determine the answer) and then write a short markdown memo cell (3–5 sentences) underneath, summarizing what you found across all five *in your own words*.

1. Which of the three feature classes has the most features, and how many does it have?
2. Which feature classes share a coordinate reference system, and which one differs? Name the CRS(s) involved.
3. Counting every field `arcpy.ListFields()` returns for each feature class (including the fields `arcpy` adds automatically, like `FID` and `Shape`), which feature class has the most total fields, and which has the fewest?
4. Is `streams_portage.geojson` included in the list `arcpy.ListFeatureClasses()` returns from this workspace? Confirm your answer with code (don't just guess) and explain, in one sentence, why it does or doesn't show up there.

5. Do all three feature classes have a field named exactly **NAME** (all upper-case)? Confirm with code; if one of them doesn't, state which one, and what its actual field name is instead.

Every one of the above questions is answerable with tools you already have: `ListFeatureClasses`, `Describe`, `ListFields`, `GetCount`, and a loop (or two). None require reading individual attribute *values* row by row.

Reuse your Task A inventory loop where it helps. For example, question 1, is nearly a one-line extension of it.

Expected output looks like: five short code-cell answers (a printed sentence or value per question is enough — no fixed format required) plus one memo paragraph underneath, 3–5 sentences, that reads as a summary rather than five restated answers in a row.

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab02_template.ipynb`), containing Task A's inventory table and its printed output, Task B's batch-buffer code and its confirmation lines, and Task C's five code-and-answer pairs plus your memo paragraph. The must all in one file, runnable top to bottom.

Point breakdown

Component	Points
Task A: Inventory script	22
Task B: Batch buffer	30
Task C: Interrogation memo	19
Style and naming	4
Total	75

“Style and naming” covers **snake_case** variable names, f-strings for all formatted output, and derived (not hand-typed) output names in Task B.

Late work follows the course late policy.